

DOCKER · DAY 02

Images, Containers & the Registry

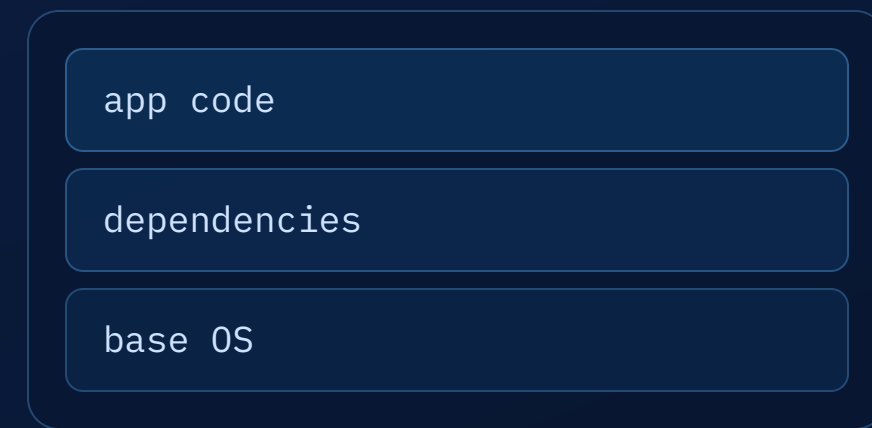
The mental models behind it all — layers, copy-on-write, image naming, and distribution.

8 mental models

animated · continuous

Image vs Container

IMAGE – read-only blueprint



nginx:1.27 · immutable

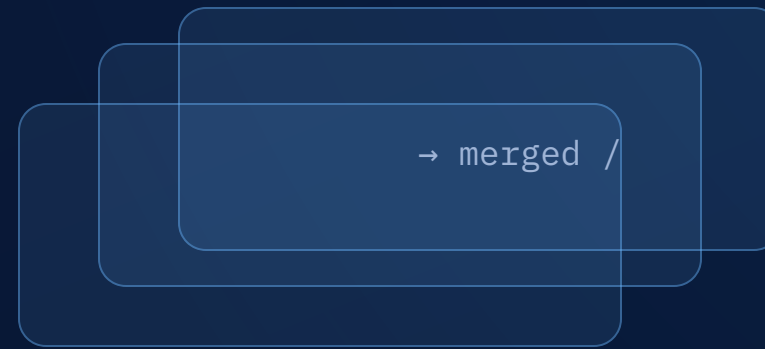
docker run

CONTAINERS – running instances



One read-only image → many cheap containers. Image = `class`, container = `object`.

The image is a stack of layers



Like transparent sheets on a projector — look down through the stack and you see one filesystem, but each sheet is stored & shipped separately.

Each layer stores only its differences — and is content-addressed by hash.

Why layers pay off: shared by hash



Sharing / dedup

Identical layers stored once. Second pull → **Already exists**.

Caching

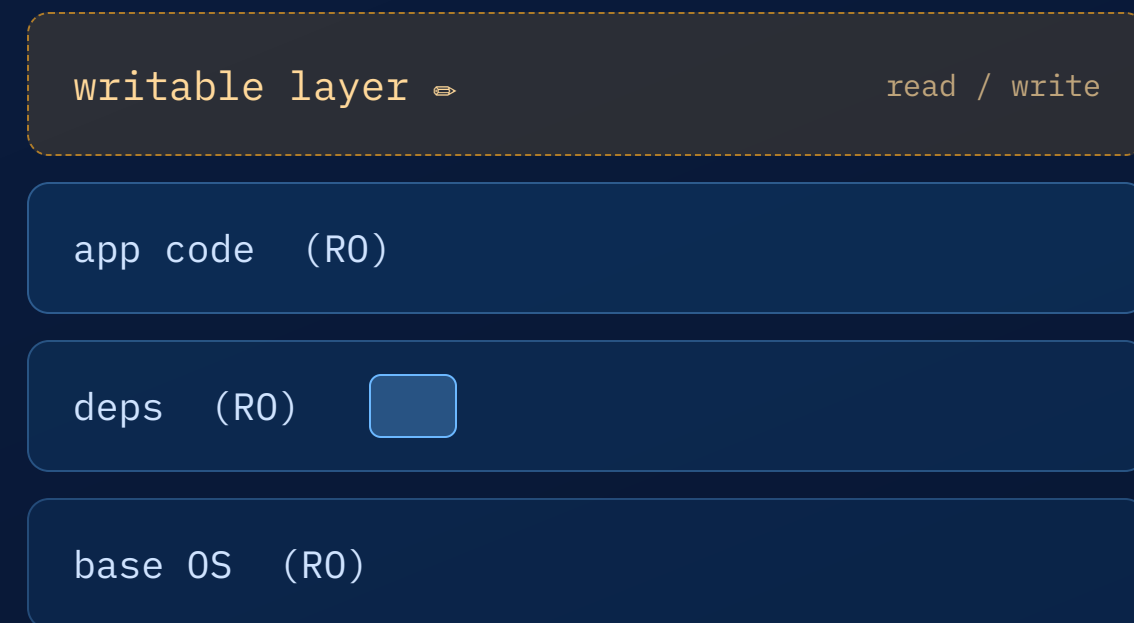
Unchanged inputs reuse the cached layer → near-instant rebuilds. Order matters.

Efficient transfer

Push / pull moves only the layers the other side lacks.

Content-addressing means a layer is stored, cached, and transferred exactly once.

Copy-on-write & the writable layer

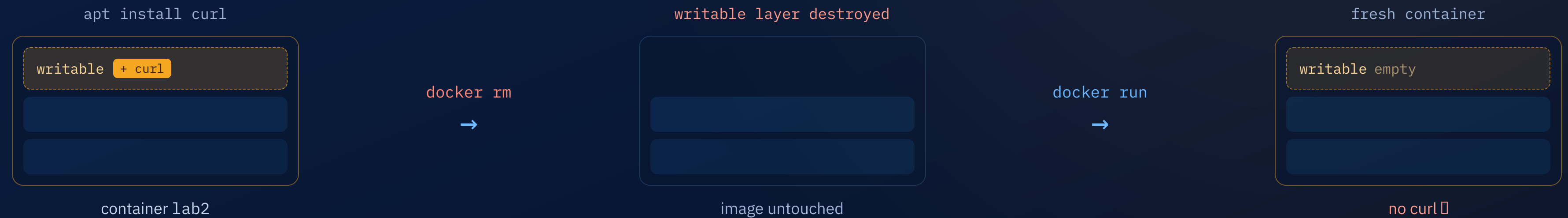


- `read` straight from the RO layer — no copy
- `modify` **copy up** into writable, then change the copy
- `new` written directly into the writable layer
- `delete` a whiteout marker hides the underlying file

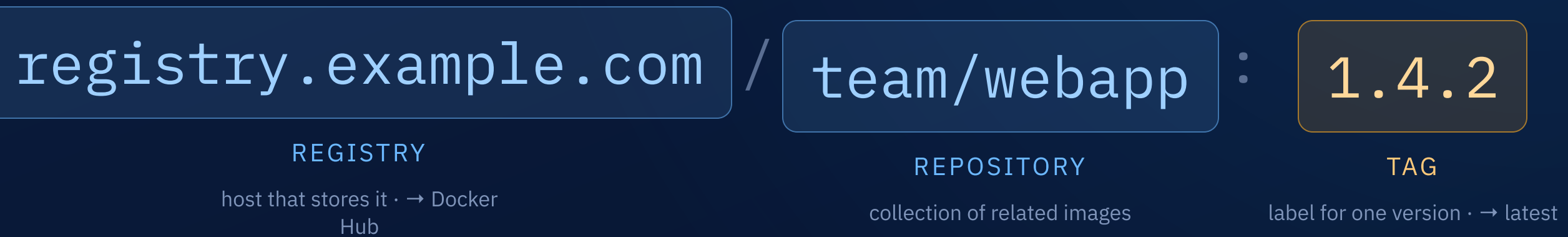
```
$ docker diff <container> # A / C / D vs image
```

Nothing is copied until something is written. The image never changes.

Why your changes vanish



Anatomy of an image reference



\$ docker pull nginx → `docker.io/library/nginx:latest`

Omit the registry → Docker Hub. Omit the tag → latest. Official images live under `library/`.

Tags move. Digests don't.

Tag — mutable

a movable sticky label

image · build #41

:1.27

image · build #42 (new)

Re-pointed any time → not a guarantee of the same bytes.

Digest — immutable

a hash of the content itself

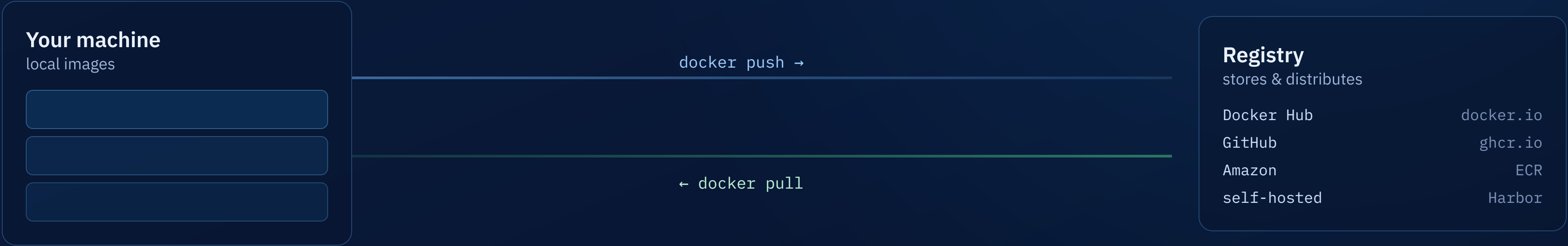


nginx@sha256:
a8f2c3...e91b

Pull by digest → exactly that image, byte for byte, forever.

⚠ **latest** is a trap — it's just the default tag, not "newest". Pin a version; in production, a digest.

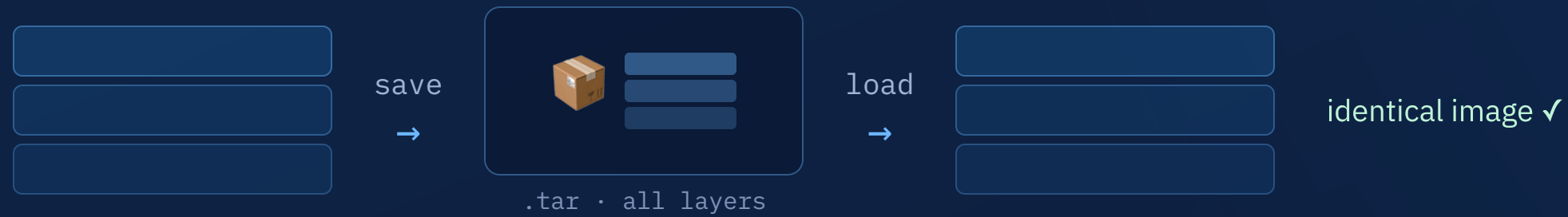
A registry is GitHub for images



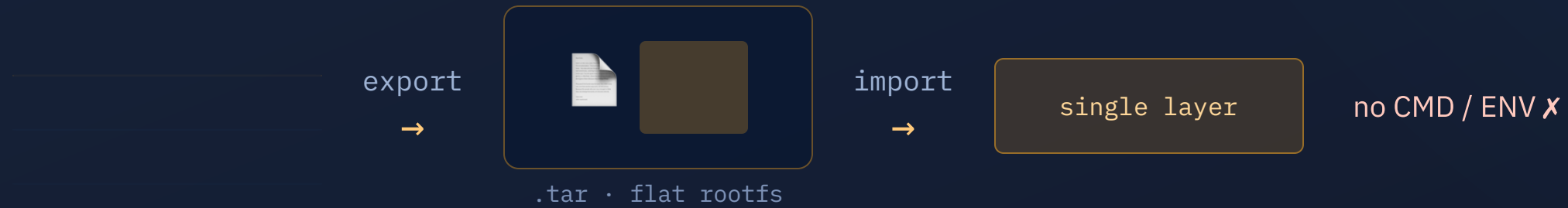
tag → login → push → pull · tag adds a name, never copies

save / load vs export / import

save / load · for IMAGES · keeps layers + history

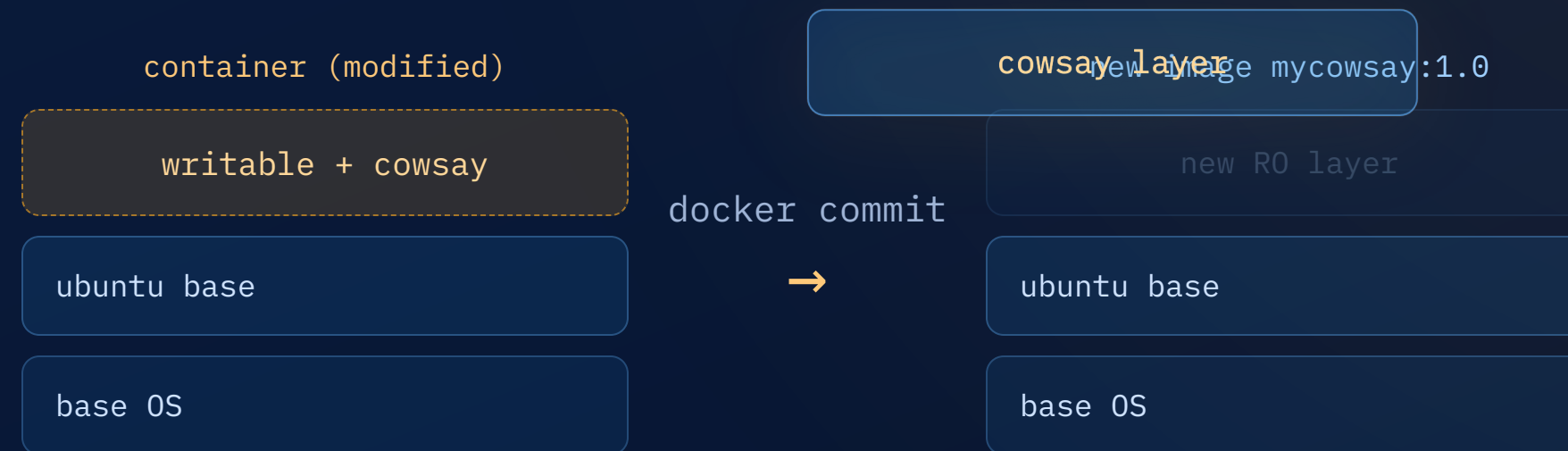


export / import · for CONTAINERS · flattens, loses history



save = images, keeps layers. export = containers, flattens & drops metadata.

commit: a writable layer freezes into an image



⚠ commit is opaque & unreproducible — a learning tool only. Real images are built from a **Dockerfile (Day 3)**.

RECAP

Ten models, two foundations

01

Image vs container

class vs object

02

Layer stack

diffs, by hash

03

Sharing & cache

stored once

04

Copy-on-write

copy up on write

05

Statelessness

layer dies with it

06

Image naming

registry/repo:tag

07

Tag vs digest

moves vs locked

08

Registry

GitHub for images

09

save vs export

keeps vs flattens

10

commit

opaque snapshot

It all rests on two ideas

layers + copy-on-write → everything else follows.

Next — Day 3: Dockerfiles (the reproducible way to build) & volumes for persistence.